

# SplitRPC: A {Control + Data} Path Splitting RPC Stack for ML Inference Serving

## 1 Paper summary

This paper looks at the so-called “RPC tax” in machine learning inference systems, especially when using accelerators like GPUs. The main idea is that models have become very fast, so now a big part of the latency is not the computation itself, but everything around it (like data movement, communication, and orchestration).

The authors point out that most existing RPC systems were designed for CPU-based services, so when they are used for ML inference they are not very efficient. There are unnecessary copies, bad use of hardware resources, and overall higher latency than needed.

To address this, they propose SplitRPC. The main idea is to separate the RPC into two parts: a control path and a data path. The CPU handles the control part, while the actual data is sent more directly to the GPU (or accelerator) using the NIC. This avoids passing everything through the CPU and reduces overhead, even if it requires some coordination.

They also introduce a queueing mechanism that allows sending and receiving at the same time, and supports dynamic batching of requests. This is important because batching is commonly used in ML inference to improve throughput, even if it can sometimes increase latency if not handled well.

The evaluation shows that SplitRPC improves performance compared to gRPC and other systems, with around 50% lower latency on average and up to 2.4x better throughput with batching. Also, it works without requiring only specialized SmartNICs, which makes it a bit more practical.

## 2 Contributions

The first contribution is the analysis of RPC overheads in ML inference systems. The paper shows that the problem is not just serialization or protocol overhead, but also how data is moved between CPU, GPU, and NIC. This is useful because it changes where optimizations should focus.

The second contribution is the design of SplitRPC itself. By splitting control and data paths, the system can use hardware resources more efficiently and reduce unnecessary work on the CPU and GPU. Also, the fact that it works with commodity NICs (not only SmartNICs) makes it more realistic to deploy, even if some assumptions are still there.

The third contribution is the queueing and batching mechanism. It allows overlapping communication and computation and improves throughput without increasing latency too much. This is important for ML serving, where both latency and throughput matter, even if in practice there is always some trade-off.

## 3 Problems

One limitation is that SplitRPC is very specific to ML inference workloads. The whole idea of splitting control and data works well here, but it might not generalize easily to other types of RPC systems. The paper kind of assumes this structure, so outside this context it could be less useful.

Another issue is the dependency on certain hardware features. The system relies on things like direct transfers between devices and efficient NIC support. Even if it doesn't

require only SmartNICs, it still assumes a fairly modern setup, which might not always be available in all environments.

A third point is that the design depends on correctly separating control and data. For the workloads in the paper this seems straightforward, but in more complex or irregular systems this could be harder. If the split is not clean, the benefits might be reduced, or the system could become more complicated than expected. Also batching, while useful, can introduce some extra coordination overhead that is not always negligible.